



Docker — Step 3

Docker Compose: running many services together — my app + Redis

Part 1 · In Progress

Docker Compose

Dockerfile vs Compose

Find-by-name

⚠ **This is a work in progress.** Step 3 is **not finished** — a lot of study remains (running it, the line-by-line walkthrough, the Redis "survive a restart" demo, networking internals, and more). This Part 1 captures the **concepts** covered so far, before running anything.

Step 3 progress: ~35% — concepts understood · hands-on not yet done

1 What Step 3 Is

So far I ran **one container at a time**, by hand. Real apps are **several services running side by side that need to talk to each other**. Step 3 is that jump, using the simplest pair: **my Node app + Redis**.

```
Step 1: docker run nginx           → ONE container
Step 2: docker run myapp          → ONE container (mine)
Step 3: docker compose up         → MANY containers, wired together, ONE command
```

What I'm building: a visit counter

Every time I open the page, the app asks Redis "what's the count?", adds 1, and shows it. The number lives in **Redis** (one container); my web code lives in **my app** (another container). Two separate boxes, cooperating.

2 The New Tool: Docker Compose

Doing this by hand would mean two `docker run` commands, manually creating a network, and remembering the right order. **Docker Compose** replaces all that with **one file and one command**.

The idea

I describe *what* I want in `docker-compose.yml`, then run `docker compose up` — Compose builds my image, pulls ready-made ones, creates a private network, and starts every service together.

Compose = the conductor

A Dockerfile is sheet music for one instrument. Compose is the conductor who brings all the musicians in together, in time, on the same stage.

3 If Compose Exists, Why Keep the Dockerfile?

Because they do **two different jobs** — Compose doesn't replace the Dockerfile, it **uses** it.

	Dockerfile	docker-compose.yml
Answers	"How do I build my app into an image?"	"How do I run several services together?"
Scope	One image	Many containers + network + config
Contains	<code>FROM</code> , <code>COPY</code> , <code>RUN</code> , <code>CMD</code>	services, ports, env vars, <code>depends_on</code>
Analogy	Recipe for one dish	The whole dinner plan

Compose CALLS the Dockerfile

In my compose file, the `web` service says `build: .` — "build this using the Dockerfile in this folder." So the Dockerfile isn't useless; Compose points straight at it. The `redis` service instead says `image: redis:7` — ready-made, no build.

```
docker-compose.yml —"build: ."→ Dockerfile —builds→ my image —runs→ web container
|
└—"image: redis:7"→ pull ready-made Redis —runs→ redis container
```

I need to...	Use
Package my own code into an image	Dockerfile
Run a ready-made image (Redis, nginx)	just <code>image:</code> — no Dockerfile
Run several services together	docker-compose.yml (builds my Dockerfile <i>and</i> pulls ready-made ones)

Why not one file for everything?

A Dockerfile can only describe **one** image — it has no concept of another service, a network, or "talk to Redis." Compose exists precisely for the multi-container coordination a Dockerfile can't express.

4 My Step 3 Files

File	Job
<code>index.js</code>	Visit-counter app — talks to Redis using the redis client
<code>package.json</code>	Now has two deps: <code>express</code> + <code>redis</code>
<code>Dockerfile</code>	Builds an image for my app only (not Redis)
<code>.dockerignore</code>	Keeps junk out of the build
<code>docker-compose.yml</code>	The new star — runs both containers together

The compose file (first look)

```
services:
  web:                                # my app
    build: .                          # build it from my Dockerfile
    ports:
      - "3000:3000"                  # open the door so I can visit it
    environment:
      REDIS_URL: redis://redis:6379  # 🖱️ tell my app where Redis is
    depends_on:
      - redis                        # start Redis first

  redis:                              # the cache
    image: redis:7                  # ready-made, don't build
```

5 The Two Big Concepts

Concept	What it means
One command runs everything	<code>docker compose up</code> starts my app <i>and</i> Redis together — no manual juggling
Services find each other by name 🌟	My app reaches Redis at <code>redis://redis:6379</code> — the word <code>redis</code> is the <i>service name</i> , not an IP

🌟 The heart of Step 3

In `index.js` : `url: "redis://redis:6379"` . That `redis` is the service name from the compose file. Docker creates a private network where services find each other **by name** automatically — no IP

addresses anywhere. This is the single most valuable real-world Docker skill, and the foundation of my end goal (frontend + backend + Redis + LiveKit).

6 Where I Am & What's Still Ahead

✓ Covered so far (this Part 1)

- What Step 3 is — one container → many cooperating
- What Docker Compose is and why it exists
- **Dockerfile vs Compose** — different jobs; Compose uses the Dockerfile
- The project files and a first look at the compose file
- The two big concepts: one command, find-by-name

📖 Still to study (NOT done yet)

- **Run it:** `docker compose up` and read the output (pull, build, network, start)
- **Line-by-line** walkthrough of every compose key (`build` , `ports` , `environment` , `depends_on` , `image`)
- **The "why /app dir is not visible" question** from last session
- **The killer demo:** restart my app, watch the count *survive* in Redis; then read the value inside the Redis container
- **The private network** — how find-by-name actually works under the hood
- `docker compose down` , `-d` background mode, logs per service, rebuilds
- **Volumes** — keeping data alive after containers are removed

My learning ladder

- ✓ **Step 0 — Docker runs**
hello-world
- ✓ **Step 1 — Ran a ready-made image (nginx)**
ports, lifecycle, Docker Hub, caching
- ✓ **Step 2 — Built my own image**
Dockerfile, build, going inside, the port puzzle
- ▶ **Step 3 — Docker Compose (IN PROGRESS)**
my app + Redis; concepts done, hands-on remaining
- 4 **Step 4 — Full multi-service stack**
frontend + backend + Redis + LiveKit

✓ One-line summary so far

Step 3 is about running **many services together** with **Docker Compose** — one file, one command. The Dockerfile still builds *my* app (Compose calls it with `build: .`), while ready-made services use

`image:` . The magic is services finding each other **by name**. *Next: actually run it and go deeper.*